

```
app_runner@vps-contabo:~/lab7$ touch file.txt
app_runner@vps-contabo:~/lab7$ nvim write_line.sh
app_runner@vps-contabo:~/lab7$ |
```

```
1 #!/bin/bash
```

```
2 echo "New line at $(date)" >> "$HOME/lab7/file.txt"
```

```
app_runner@vps-contabo:~/lab7$ chmod +x write_line.sh
app_runner@vps-contabo:~/lab7$ ls -l write_line.sh
-rwxrwxr-x 1 app_runner app_runner 58 Sep 17 18:02 write_line.sh
app_runner@vps-contabo:~/lab7$ |
```

```
app_runner@vps-contabo:~/lab7$ crontab -e
no crontab for app_runner - using an empty one
crontab: installing new crontab
app_runner@vps-contabo:~/lab7$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h dom mon dow   command
* * * * * /home/app_runner/lab7/write_line.sh
app_runner@vps-contabo:~/lab7$ |
```

```
app_runner@vps-contabo:~/lab7$ cat "$HOME/lab7/file.txt"
New line at Thu Sep 17 18:05:17 CEST 2026
New line at Thu Sep 17 18:05:33 CEST 2026
New line at Thu Sep 17 18:05:59 CEST 2026
New line at Thu Sep 17 18:12:01 CEST 2026
app_runner@vps-contabo:~/lab7$ |
```

Task: try to create cron that runs python script with some functionality, e.g. sending email to server owner

```
app_runner@vps-contabo:~/lab7$ nvim cron_task.py
app_runner@vps-contabo:~/lab7$ |
```

```
1 from datetime import datetime
2
3 with open("/home/app_runner/lab7/python_log.txt", "a") as file:
4     file.write(f"Python cron task executed at {datetime.now()}\n")
```

```
app_runner@vps-contabo:~/lab7$ crontab -e
crontab: installing new crontab
app_runner@vps-contabo:~/lab7$ crontab -l
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
* * * * * /usr/bin/python3 /home/app_runner/lab7/cron_task.py
app_runner@vps-contabo:~/lab7$ cat /home/app_runner/lab7/python_log.txt
Python cron task executed at 2026-09-17 18:18:09.693464
Python cron task executed at 2026-09-17 18:18:22.846075
Python cron task executed at 2026-09-17 18:19:01.222970
app_runner@vps-contabo:~/lab7$ |
```